

PRACTICAL RECORD

Spring Boot Student Management REST API

Name: S Svava Haasini

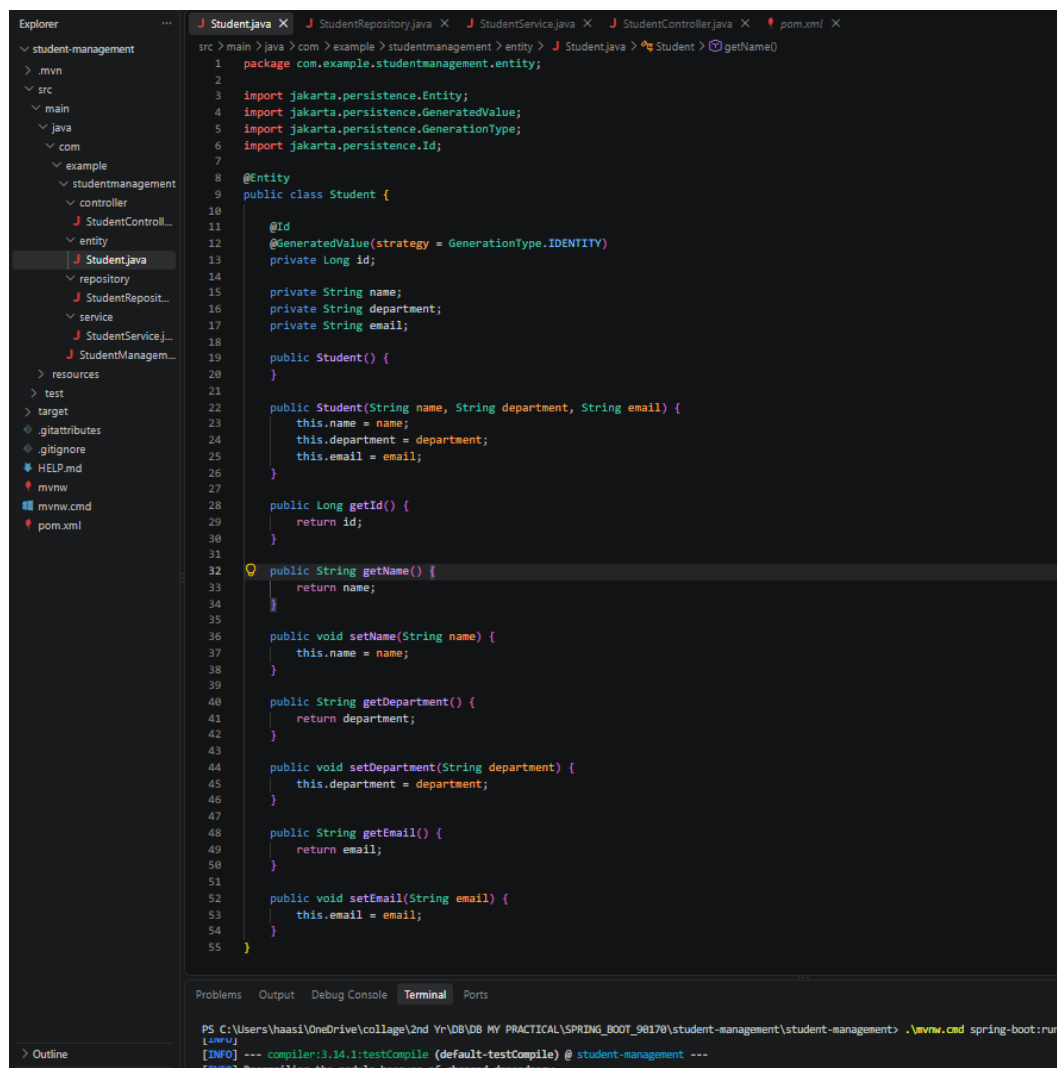
Roll Number: 2520090170

Aim:

To develop a Spring Boot REST API for Student Management system with MySQL database integration, demonstrating CRUD operations (Create, Read, Update, Delete) using HTTP methods.

1. Student Entity

The Student entity class contains id, name, department, and email attributes with automatic ID generation using GenerationType.IDENTITY.



```
src > main > java > com > example > studentmanagement > entity > J Student.java > Student > getName()
1  package com.example.studentmanagement.entity;
2
3  import jakarta.persistence.Entity;
4  import jakarta.persistence.GeneratedValue;
5  import jakarta.persistence.GenerationType;
6  import jakarta.persistence.Id;
7
8  @Entity
9  public class Student {
10
11      @Id
12      @GeneratedValue(strategy = GenerationType.IDENTITY)
13      private Long id;
14
15      private String name;
16      private String department;
17      private String email;
18
19      public Student() {
20      }
21
22      public Student(String name, String department, String email) {
23          this.name = name;
24          this.department = department;
25          this.email = email;
26      }
27
28      public Long getId() {
29          return id;
30      }
31
32      public String getName() {
33          return name;
34      }
35
36      public void setName(String name) {
37          this.name = name;
38      }
39
40      public String getDepartment() {
41          return department;
42      }
43
44      public void setDepartment(String department) {
45          this.department = department;
46      }
47
48      public String getEmail() {
49          return email;
50      }
51
52      public void setEmail(String email) {
53          this.email = email;
54      }
55  }
```

Problems Output Debug Console Terminal Ports

PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\SPRING_BOOT_90170\student-management\student-management> .\mvnw.cmd spring-boot:run

[INFO] --- compiler:3.14.1:testCompile (default-testCompile) @ student-management ---

[INFO] Recompiling the module because of changed dependency.

Figure 1: Student.java - Entity Class

2. Student Repository

The StudentRepository interface extends JpaRepository to provide database operations such as save(), findAll(), findById(), and deleteById().

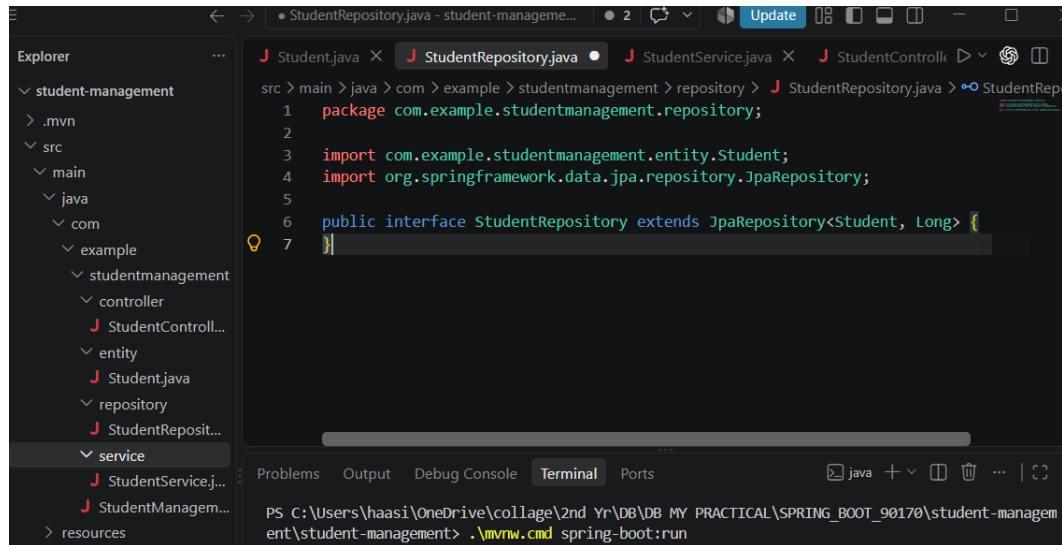


Figure 2: StudentRepository.java - Repository Interface

3. Student Service

The StudentService class contains business logic for creating, retrieving, updating, and deleting student records through the repository.

```
1 package com.example.studentmanagement.service;
2
3 import com.example.studentmanagement.entity.Student;
4 import com.example.studentmanagement.repository.StudentRepository;
5 import org.springframework.stereotype.Service;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 @Service
11 public class StudentService {
12
13     private final StudentRepository repository;
14
15     public StudentService(StudentRepository repository) {
16         this.repository = repository;
17     }
18
19     public Student create(Student student) {
20         return repository.save(student);
21     }
22
23     public List<Student> getAll() {
24         return repository.findAll();
25     }
26
27     public Optional<Student> getById(Long id) {
28         return repository.findById(id);
29     }
30
31     public Student update(Long id, Student data) {
32
33         Student student = repository.findById(id)
34             .orElseThrow(() -> new RuntimeException(message: "Student not found"));
35
36         student.setName(data.getName());
37         student.setDepartment(data.getDepartment());
38         student.setEmail(data.getEmail());
39
40         return repository.save(student);
41     }
42
43     public void delete(Long id) {
44         repository.deleteById(id);
45     }
46 }
```

PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\SPRING_BOOT_90170\student-managem
ent\student-management> .\mvnw.cmd spring-boot:run

Figure 3: StudentService.java - Service Layer

4. Student REST Controller

The StudentController provides REST endpoints for POST, GET, PUT, and DELETE operations on the /api/students path.

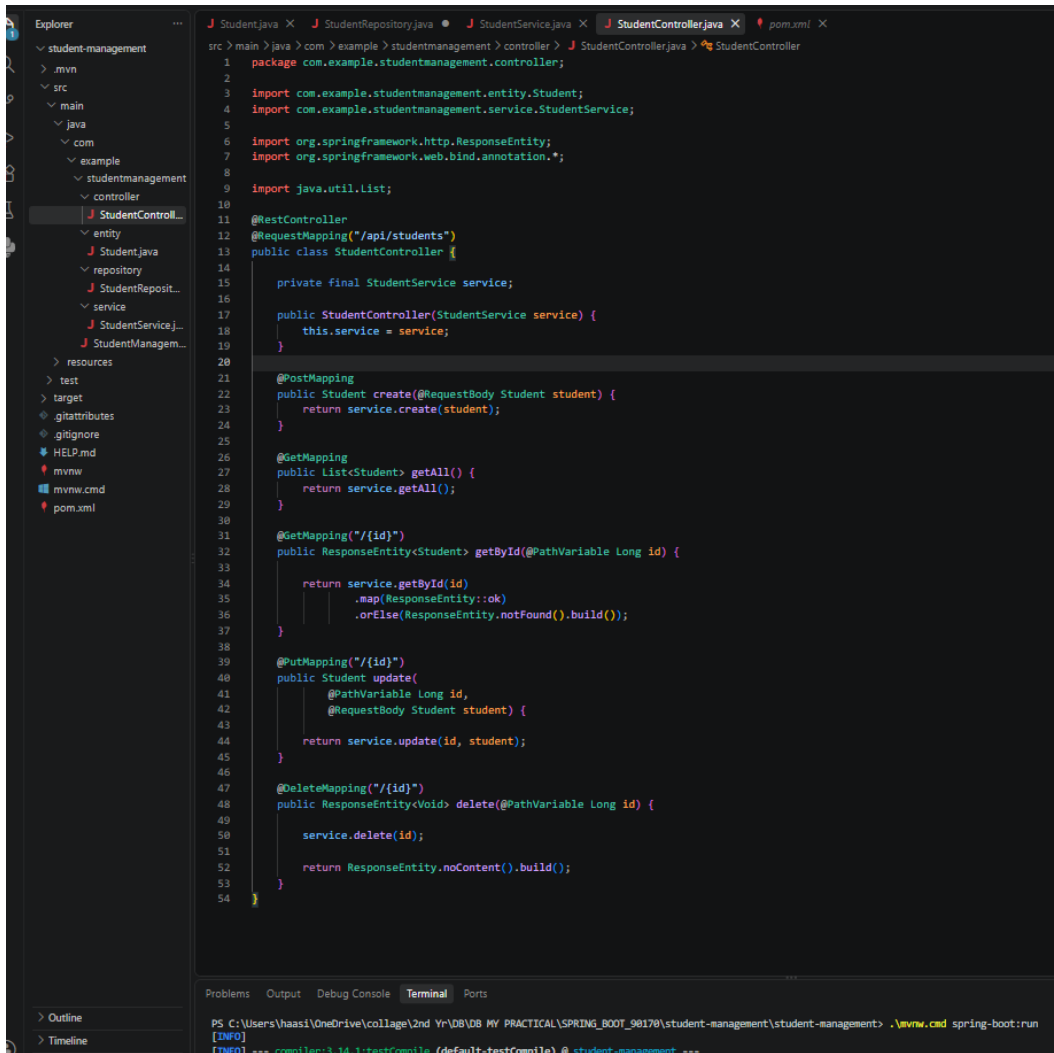


Figure 4: StudentController.java - REST Controller

5. MySQL Configuration

The application.properties file contains MySQL database URL, credentials, Hibernate configuration with ddl-auto=update, and server port 8080.

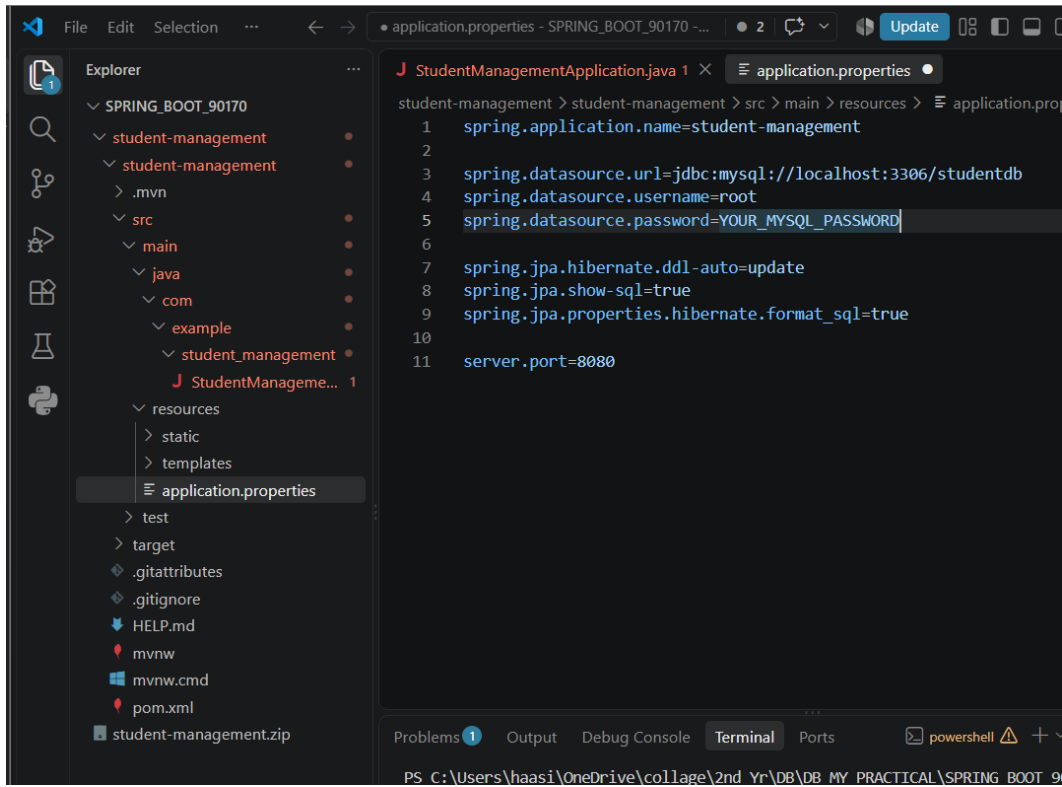


Figure 5: application.properties - Database Configuration

6. POST – Create Student

The POST request creates a new student record with name, department, and email and stores it in the MySQL database.

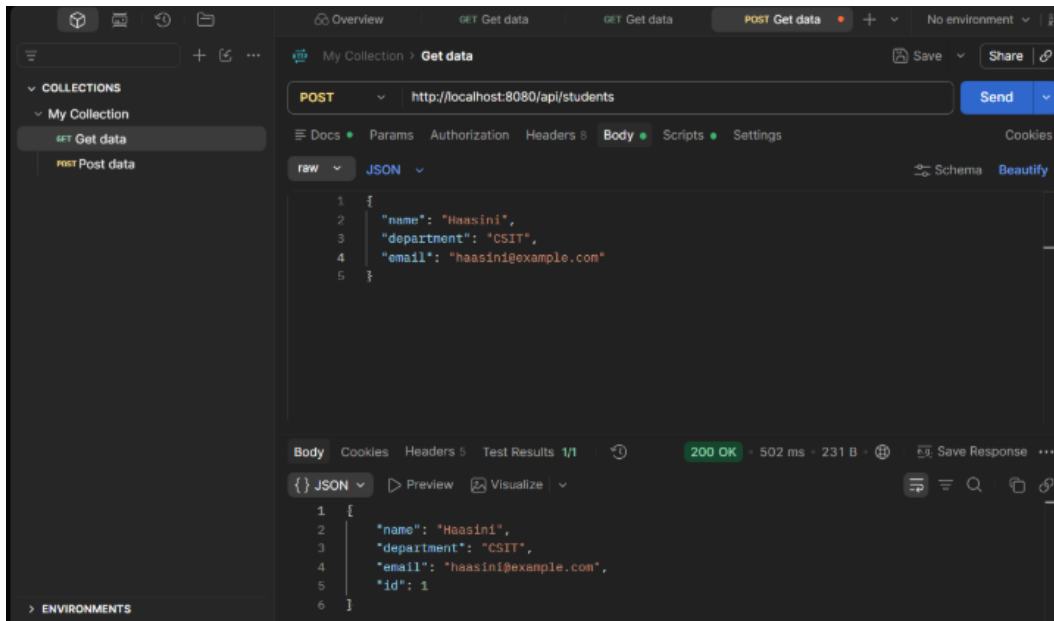


Figure 6: POST Request – Create Student

7. GET – Read Students

The GET request retrieves all student records stored in the database through the Spring Boot REST API.

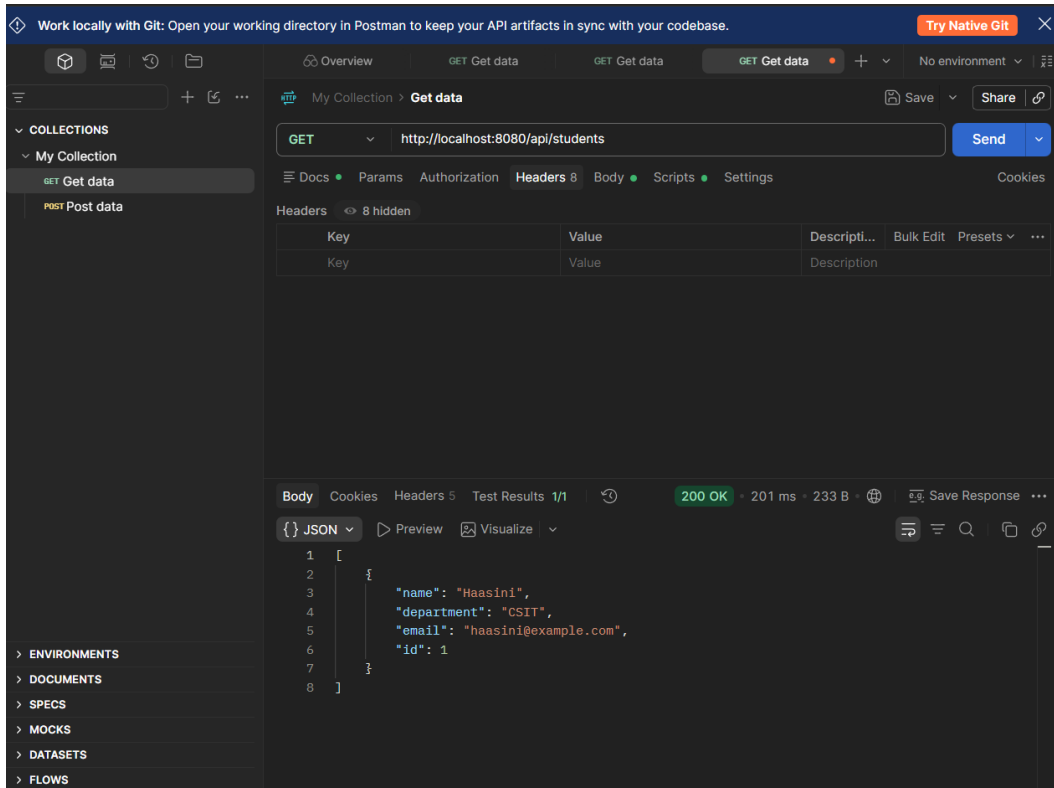


Figure 7: GET Request – View Students

8. PUT – Update Student

The PUT request updates the existing student record with new name, department, and email information.

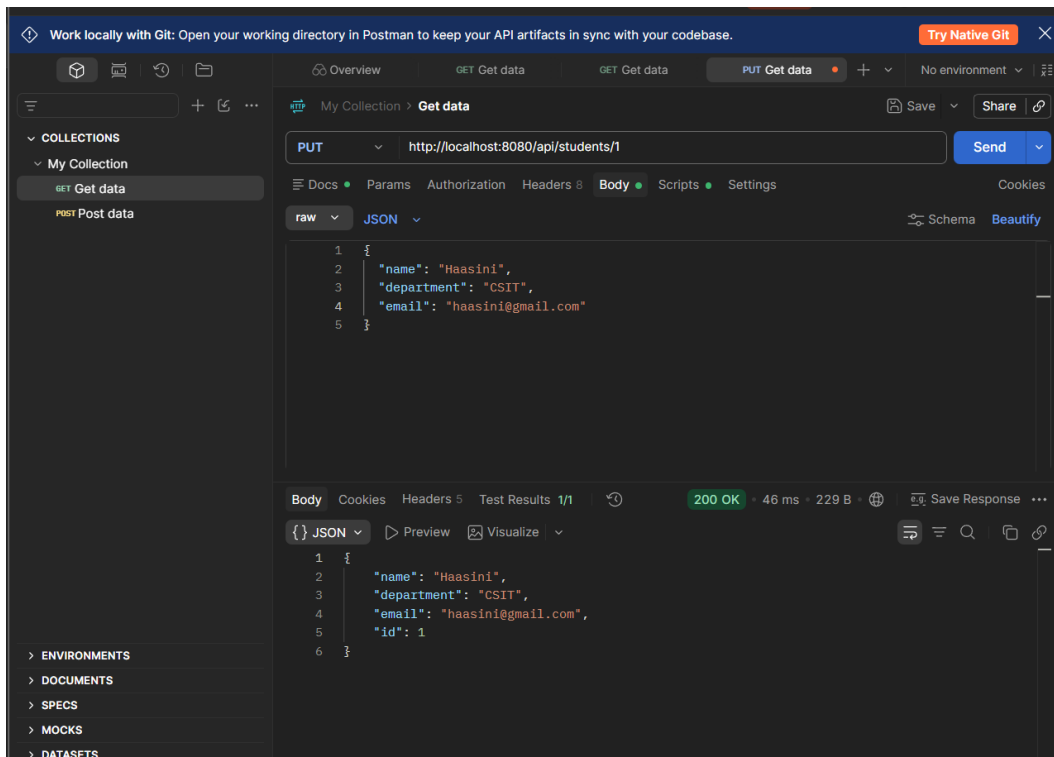


Figure 8: PUT Request – Update Student

9. Verify Data in MySQL

The student data sent through the Spring Boot REST API is verified in MySQL Workbench using `SELECT * FROM student` query on the studentdb database.

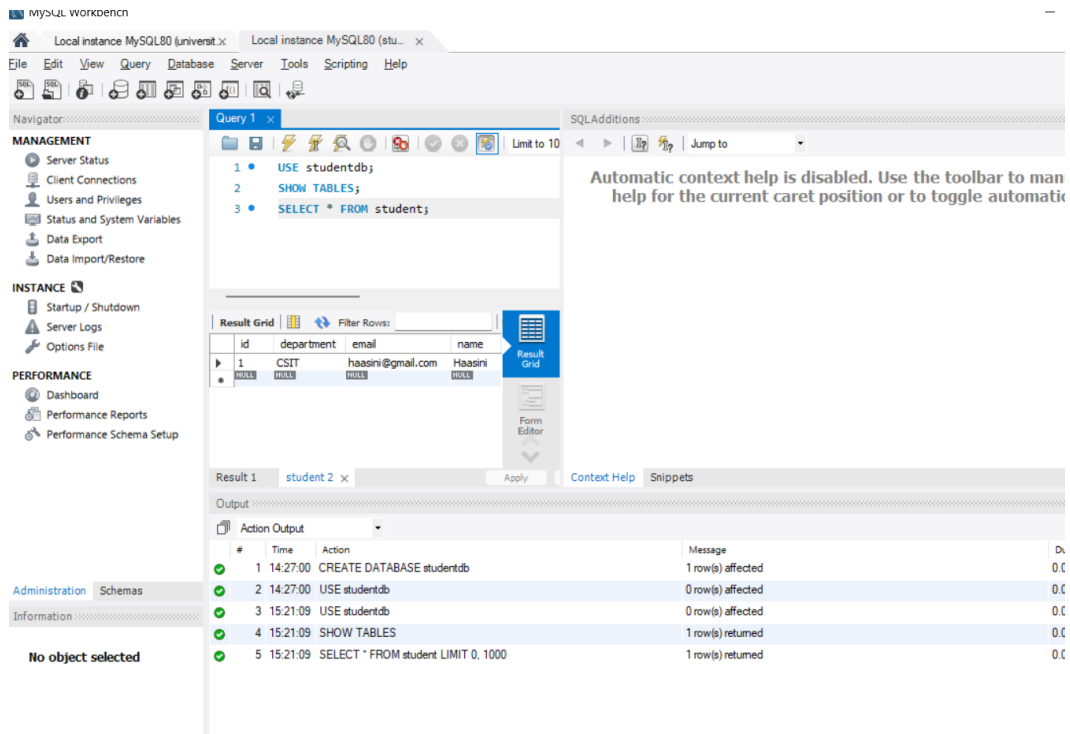


Figure 9: Student Data Stored in MySQL

10. DELETE – Delete Student

The DELETE request removes the selected student record from the database and returns a 204 No Content response.

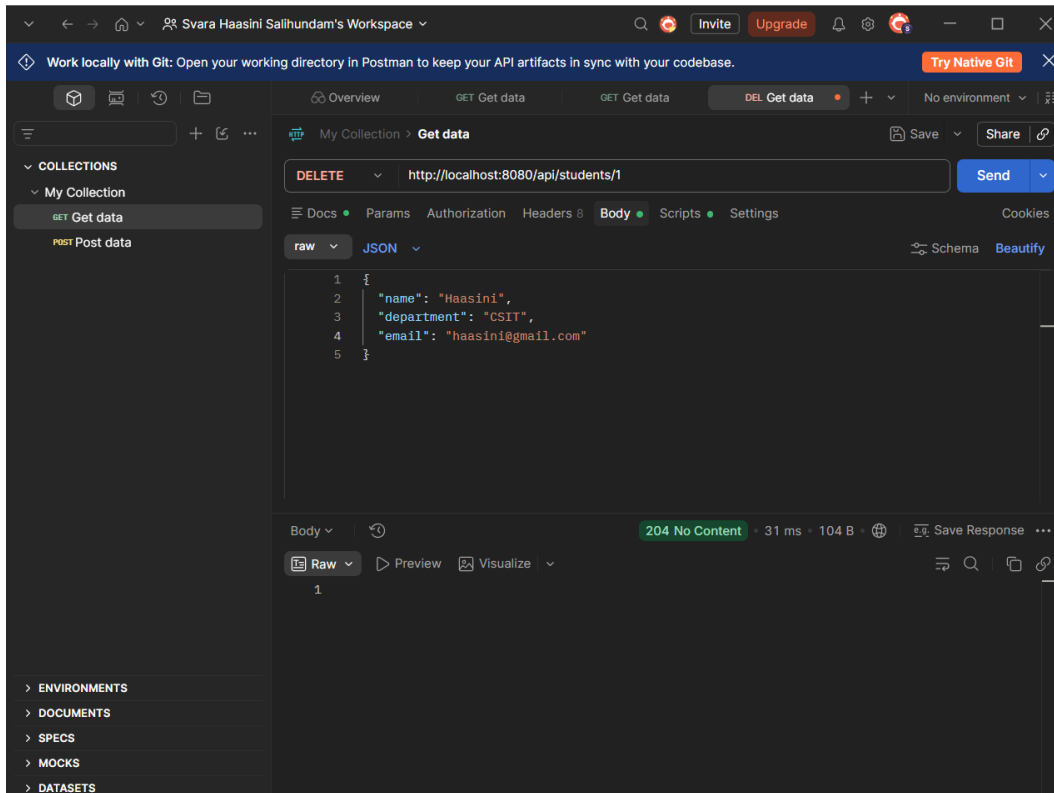


Figure 10: DELETE Request – Delete Student

11. Verify Data After Deletion

The MySQL database is queried again to verify that the deleted student record has been removed successfully.

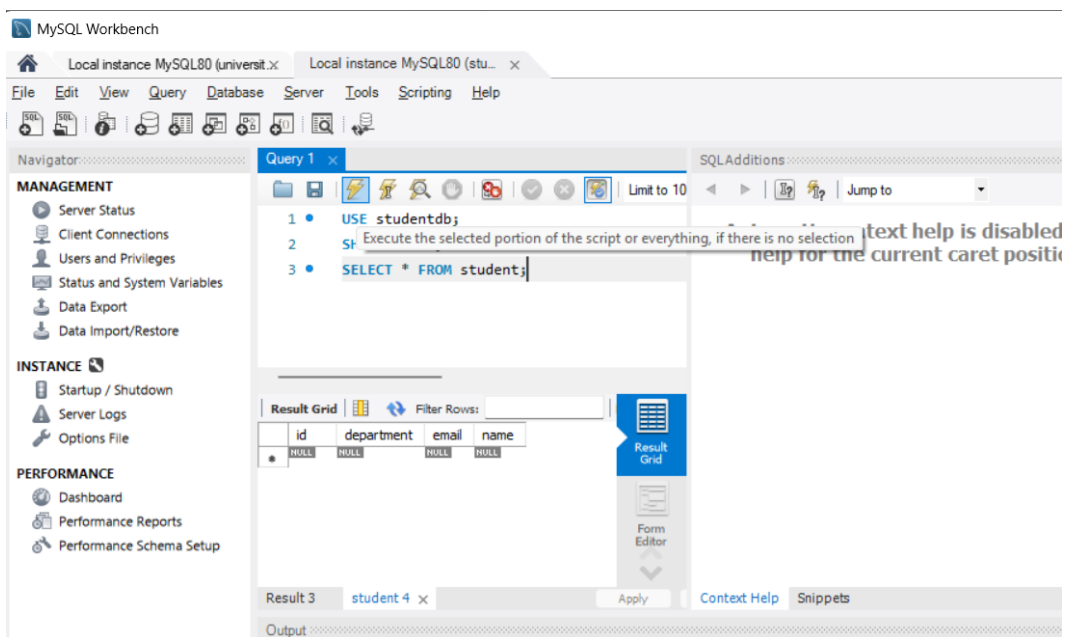


Figure 11: Database Verification After Deletion

Result:

The Spring Boot Student Management REST API was successfully created and tested using POST, GET, PUT, and DELETE operations, and the data was successfully integrated with the MySQL database.